PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Curso de Engenharia de Software · Disciplina: Projeto de Software
Profa. Milena Menezes Adão

Laboratório 3 · Sprint 1 (Lab03S01)

Sistema de Moeda Estudantil

Modelagem do sistema: casos de uso, histórias de usuário, diagrama de classes e diagrama de componentes

Aluno: Lucas Lima Magalhães Lafetá Chaves
Repositório: github.com/lucaslmchaves/sistema-moeda-estudantil
Data: outubro de 2026

Sumário

1 Visão geral e decisões de modelagem

2 Diagrama de casos de uso

3 Histórias de usuário

4 Diagrama de classes

5 Diagrama de componentes (arquitetura MVC)

6 Rastreabilidade entre os modelos

7 Próximas etapas e referências

1. Visão geral e decisões de modelagem

1.1 Descrição do sistema

O Sistema de Moeda Estudantil estimula o reconhecimento do mérito dos estudantes por meio de uma moeda virtual. Os professores distribuem moedas aos seus alunos, por exemplo por bom comportamento ou participação em aula, e os alunos trocam essas moedas por produtos e descontos oferecidos por empresas parceiras.

O aluno se cadastra informando nome, e-mail, CPF, RG, endereço, instituição de ensino e curso. As instituições participantes e os professores (nome, CPF e departamento) já vêm pré-cadastrados. A cada semestre cada professor recebe mil moedas, e o que não for distribuído se acumula no saldo. Para enviar moedas, o professor precisa de saldo suficiente e deve informar o aluno e o motivo do reconhecimento, uma mensagem obrigatória. O aluno é avisado por e-mail ao receber moedas.

Professores e alunos consultam o extrato da conta, com o saldo e as transações. As empresas parceiras se cadastram e oferecem vantagens, cada uma com descrição, foto e custo em moedas. Ao trocar moedas por uma vantagem, o valor é descontado do saldo do aluno e um e-mail com um código de cupom, gerado pelo sistema, é enviado ao aluno e à empresa, para conferir a troca. Todos os usuários acessam o sistema com login e senha.

1.2 Escopo da Sprint 1

Esta entrega reúne a modelagem pedida para o Lab03S01 no enunciado:

- **Diagrama de casos de uso** (seção 2);
- **Histórias de usuário** (seção 3);
- **Diagrama de classes** (seção 4);
- **Diagrama de componentes**, na arquitetura MVC exigida (seção 5).

Todos os diagramas estão versionados no repositório, em `docs/diagramas/`, com o prefixo `sprint1-`. As versões das próximas sprints entrarão como novos arquivos, sem apagar as anteriores.

1.3 Decisões de modelagem

O enunciado deixa alguns pontos em aberto. A tabela registra o que foi decidido e por quê.

Decisão	Escolha	Justificativa
Instituições e professores	Pré-cadastrados, sem caso de uso de cadastro	O enunciado diz que as instituições já estão no sistema e que a instituição envia a lista de professores.
Semestre	Evento do sistema, não um ator	O crédito de 1.000 moedas é automático. O caso de uso "Receber Moedas do Semestre" fica ligado ao Professor, que é quem recebe, sem criar um ator novo.
Saldo de moedas	Classe `Conta`	Evita repetir saldo e operações em Aluno e Professor e dá uma única responsabilidade à regra de saldo.

Decisão	Escolha	Justificativa
Transação	Hierarquia `Transacao` → `EnvioMoedas` e `ResgateVantagem`	Cada tipo guarda só os atributos que usa, sem campos nulos e sem enumeração de tipo.
Cupom	Classe própria, em composição com `ResgateVantagem`	O código é gerado pelo sistema e enviado nos dois e-mails; um resgate gera exatamente um cupom.
E-mail	Interface `NotificadorEmail`	O serviço de moedas depende de uma abstração e o provedor de e-mail pode ser trocado sem alterar a regra de negócio.
Empresa parceira	Inclui `cnpj`	Identificação da empresa. O enunciado não cita o CNPJ, é uma suposição do projeto.
Endereço e curso	Atributos `String`	O enunciado os trata como dados simples do cadastro do aluno.
Atualizar e excluir cadastro	Casos de uso de aluno e empresa	As Sprints 2 e 3 pedem o CRUD completo de aluno e de empresa parceira.
Arquitetura	MVC, com a persistência em camada própria	Exigência do enunciado.

2. Diagrama de casos de uso

2.1 Atores

Ator	Tipo	Papel no sistema
Aluno	Pessoa	Participa do sistema de mérito, recebe moedas dos professores e as troca por vantagens.
Professor	Pessoa	Distribui moedas aos alunos e recebe 1.000 moedas por semestre.
Empresa Parceira	Organização	Cadastra as vantagens que oferece em troca de moedas.
Serviço de E-mail	Sistema externo	Envia a notificação de moedas recebidas e os e-mails de cupom.

As pessoas e a empresa ficam à esquerda do sistema. O serviço de e-mail é um ator secundário (sistema externo) e fica à direita, ligado apenas aos casos de uso que dependem dele.

2.2 Relacionamentos

- **Autenticar-se** é um caso de uso próprio, ligado a Aluno, Professor e Empresa Parceira, que são os atores que fazem login.
- **«include»** em dois pontos, pois a ação sempre dispara a seguinte: *Enviar Moedas a um Aluno* inclui *Notificar Aluno sobre Moedas Recebidas*, e *Trocar Moedas por Vantagem* inclui *Enviar Cupom de Resgate*.
- Não há «extend», porque nenhuma extensão depende de uma condição real.

2.3 Diagrama de casos de uso

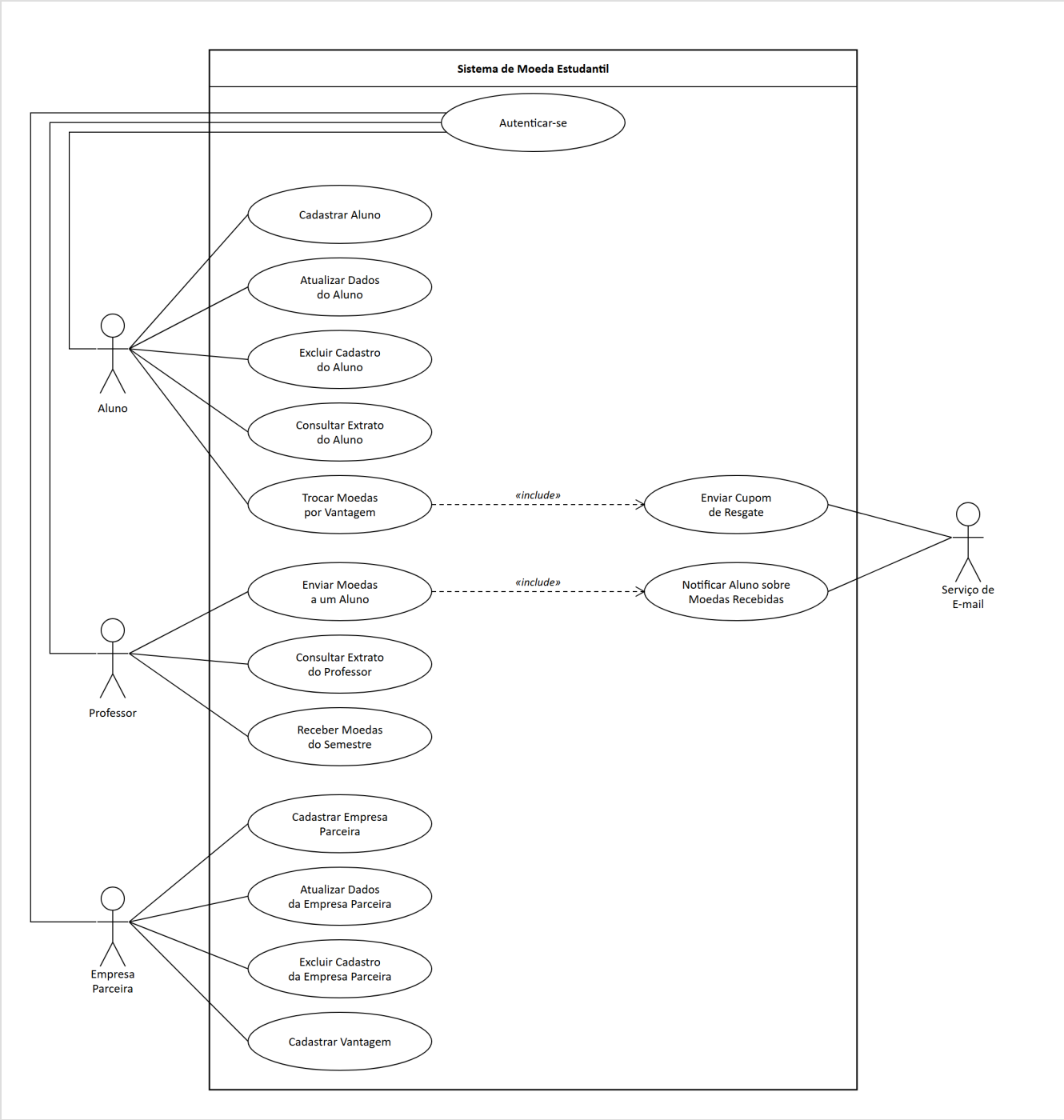


Figura 1 – Diagrama de casos de uso do Sistema de Moeda Estudantil

2.4 Descrição dos casos de uso

Há um caso de uso para cada história de usuário da seção 3, com o mesmo nome.

Caso de uso	Ator	Descrição
Autenticar-se	Aluno, Professor, Empresa Parceira	Acesso ao sistema com login e senha; é exigido em todas as funcionalidades.
Cadastrar Aluno	Aluno	Cadastro com nome, e-mail, CPF, RG, endereço, curso, instituição (escolhida entre as pré-cadastradas), login e senha.

Caso de uso	Ator	Descrição
Atualizar Dados do Aluno	Aluno	Alteração dos dados cadastrais do próprio aluno.
Excluir Cadastro do Aluno	Aluno	Remoção do cadastro do próprio aluno.
Consultar Extrato do Aluno	Aluno	Saldo atual, recebimentos de moedas e trocas por vantagens.
Trocar Moedas por Vantagem	Aluno	Escolha de uma vantagem, desconto do custo no saldo e geração do cupom. Inclui “Enviar Cupom de Resgate”.
Enviar Moedas a um Aluno	Professor	Envio de moedas com saldo suficiente e mensagem obrigatória. Inclui “Notificar Aluno sobre Moedas Recebidas”.
Consultar Extrato do Professor	Professor	Saldo atual e envios de moedas realizados.
Receber Moedas do Semestre	Professor	Crédito automático de 1.000 moedas por semestre, somado ao saldo corrente.
Cadastrar Empresa Parceira	Empresa Parceira	Cadastro com nome, e-mail, CNPJ, login e senha.
Atualizar Dados da Empresa Parceira	Empresa Parceira	Alteração dos dados cadastrais da própria empresa.
Excluir Cadastro da Empresa Parceira	Empresa Parceira	Remoção do cadastro da própria empresa.
Cadastrar Vantagem	Empresa Parceira	Cadastro de vantagem com descrição, foto e custo em moedas.
Notificar Aluno sobre Moedas Recebidas	Serviço de E-mail	E-mail ao aluno sempre que recebe moedas.
Enviar Cupom de Resgate	Serviço de E-mail	E-mail com o código do cupom ao aluno e à empresa, para conferir a troca.

3. Histórias de usuário

As histórias seguem o formato *Como* (papel), *quero* (ação), *para* (benefício) e estão agrupadas por ator. Os itens de cada história são as regras de negócio do enunciado, e é deles que saem as constantes, as cardinalidades e as operações do diagrama de classes.

Todos os perfis

Autenticar-se

***Como** aluno, professor ou empresa parceira, **quero** entrar no sistema com meu login e senha, **para** acessar as funcionalidades do meu perfil.*

- Alunos, professores e empresas parceiras têm login e senha cadastrados.
- Nenhuma funcionalidade é acessível sem autenticação.

Aluno

Cadastrar aluno

***Como** aluno, **quero** me cadastrar no sistema de mérito, **para** receber moedas dos meus professores e trocá-las por vantagens.*

- O aluno informa nome, e-mail, CPF, RG, endereço, curso, login e senha.
- A instituição de ensino é escolhida entre as instituições já pré-cadastradas.

Atualizar dados do aluno

***Como** aluno, **quero** atualizar meus dados cadastrais, **para** mantê-los corretos.*

- Só o próprio aluno autenticado altera os seus dados.

Excluir cadastro do aluno

***Como** aluno, **quero** excluir meu cadastro, **para** deixar de participar do sistema de mérito.*

- Só o próprio aluno autenticado exclui o seu cadastro.

Consultar extrato do aluno

***Como** aluno, **quero** consultar o extrato da minha conta, **para** acompanhar meu saldo e minhas movimentações.*

- O extrato mostra o total de moedas que o aluno ainda possui.
- O extrato lista os recebimentos de moedas e as trocas por vantagens.

Trocar moedas por vantagem

***Como** aluno, **quero** trocar minhas moedas por uma vantagem cadastrada, **para** aproveitar o reconhecimento que recebi.*

- O aluno seleciona uma das vantagens cadastradas no sistema.
- O saldo do aluno precisa cobrir o custo da vantagem, e o valor é descontado do saldo.
- O sistema gera um código para a troca.
- Um e-mail de cupom, com o código, é enviado ao aluno para a troca presencial.
- Um e-mail, com o mesmo código, é enviado à empresa parceira para conferir a troca.

Professor

Enviar moedas a um aluno

***Como** professor, **quero** enviar moedas a um aluno, **para** reconhecer seu bom comportamento ou sua participação em aula.*

- O professor precisa ter saldo suficiente para o montante enviado.
- O professor indica o aluno que recebe as moedas.
- O motivo do reconhecimento é uma mensagem aberta e obrigatória.
- O aluno é notificado por e-mail ao receber as moedas.

Consultar extrato do professor

***Como** professor, **quero** consultar o extrato da minha conta, **para** acompanhar meu saldo e as moedas que já enviei.*

- O extrato mostra o total de moedas que o professor ainda possui.
- O extrato lista os envios de moedas.

Receber moedas do semestre

***Como** professor, **quero** receber moedas a cada semestre, **para** ter o que distribuir aos meus alunos.*

- A cada semestre o sistema credita 1.000 moedas ao professor.
- O total é acumulável: as moedas não distribuídas permanecem no saldo e as 1.000 novas são somadas a ele.

Empresa Parceira

Cadastrar empresa parceira

***Como** empresa parceira, **quero** me cadastrar no sistema, **para** oferecer vantagens aos alunos.*

- A empresa informa nome, e-mail, CNPJ, login e senha.

Atualizar dados da empresa parceira

***Como** empresa parceira, **quero** atualizar meus dados cadastrais, **para** mantê-los corretos.*

- Só a própria empresa autenticada altera os seus dados.

Excluir cadastro da empresa parceira

***Como** empresa parceira, **quero** excluir meu cadastro, **para** encerrar a parceria com o sistema.*

- Só a própria empresa autenticada exclui o seu cadastro.

Cadastrar vantagem

***Como** empresa parceira, **quero** cadastrar as vantagens que ofereço, **para** que os alunos possam trocá-las por moedas.*

- A vantagem tem descrição, foto do produto e custo em moedas.
- A vantagem pertence à empresa que a cadastrou.

Serviço de e-mail (sistema externo)

Notificar aluno sobre moedas recebidas

***Como** serviço de e-mail, **quero** ser acionado quando um professor envia moedas, **para** avisar o aluno por e-mail.*

- O e-mail é enviado ao aluno sempre que ele recebe moedas.

Enviar cupom de resgate

***Como** serviço de e-mail, **quero** ser acionado quando um aluno resgata uma vantagem, **para** enviar o cupom ao aluno e a conferência à empresa parceira.*

- O aluno e a empresa parceira recebem, cada um, um e-mail.
- Os dois e-mails incluem o mesmo código gerado pelo sistema.

Fora do escopo

- Instituições de ensino e professores já vêm pré-cadastrados no sistema, por isso não há história de cadastro para eles.

4. Diagrama de classes

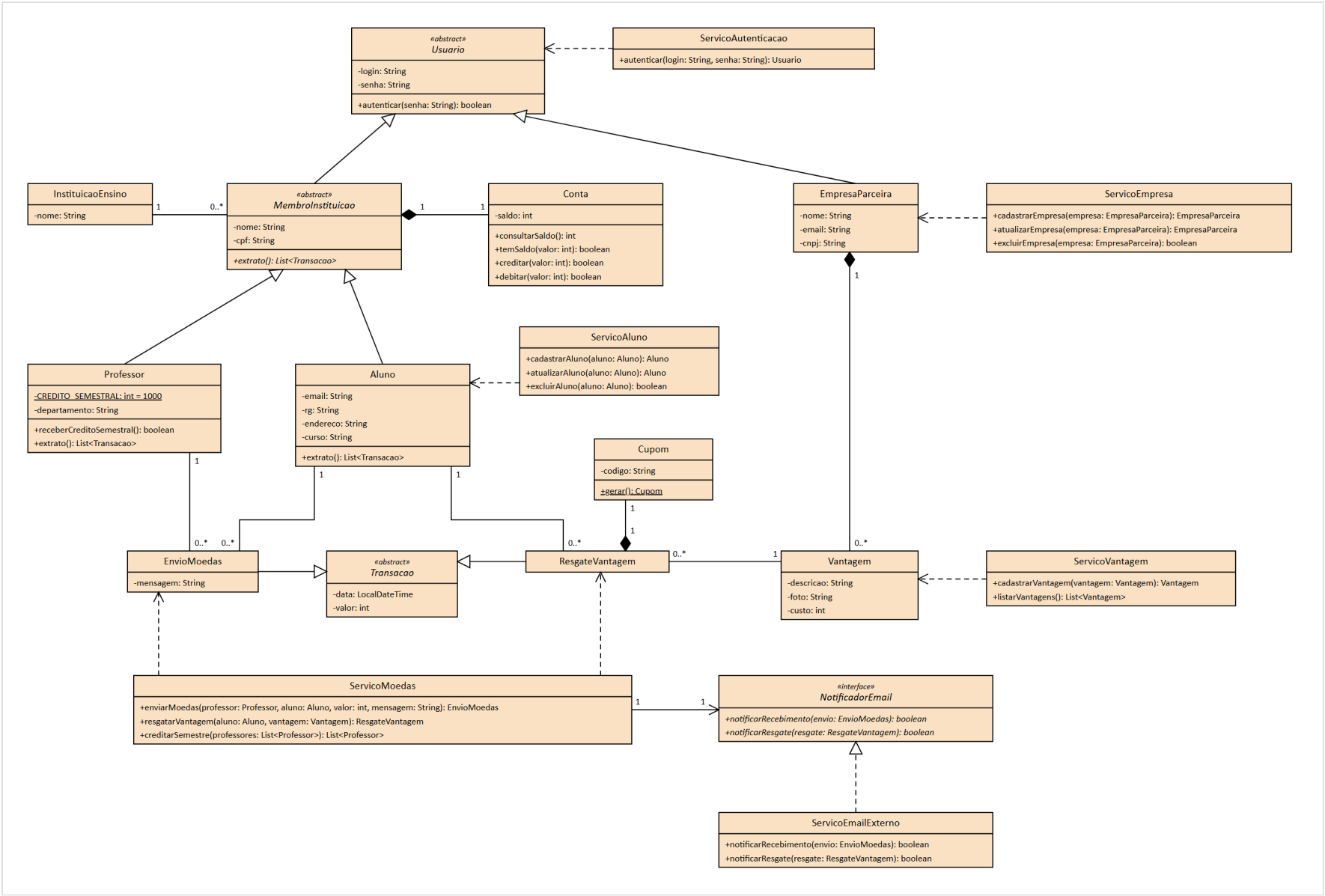


Figura 2 – Diagrama de classes (só classes de domínio, serviços, interface e as operações públicas de negócio)

O diagrama de classes é compacto: mostra apenas classes de modelo, serviços de domínio e a interface de notificação, com atributos e operações públicas de negócio. Todas as operações têm retorno declarado (nenhuma devolve `void`), e não aparecem construtores, getters e setters, métodos privados, exceções nem classes de infraestrutura.

4.1 Classes e responsabilidades

Classe	Responsabilidade
`Usuario` (abstrata)	Credenciais de acesso (login e senha) e verificação da senha.
`MembroInstituicao` (abstrata)	Dados comuns a quem pertence a uma instituição (nome e CPF), a conta de moedas e o extrato, que cada subclasse implementa.
`Aluno`	Dados do cadastro (e-mail, RG, endereço, curso). O extrato reúne recebimentos e trocas.
`Professor`	Departamento e crédito semestral (constante `CREDITO_SEMESTRAL = 1000`). O extrato reúne os envios.
`EmpresaParceira`	Dados da empresa (nome, e-mail, CNPJ). Também é um `Usuario`, mas não pertence a instituição.
`InstituicaoEnsino`	Instituição pré-cadastrada à qual alunos e professores pertencem.
`Conta`	Saldo de moedas: consultar, verificar suficiência, creditar e debitar.
`Transacao` (abstrata)	Data e valor de toda movimentação de moedas.
`EnvioMoedas`	Envio de um professor a um aluno, com a mensagem obrigatória.
`ResgateVantagem`	Troca de moedas de um aluno por uma vantagem; possui um cupom.
`Cupom`	Código gerado pelo sistema, enviado ao aluno e à empresa.
`Vantagem`	Descrição, foto e custo em moedas; pertence a uma empresa.
`ServicoAutenticacao`	Valida login e senha e devolve o usuário autenticado.
`ServicoAluno`	Cadastro, atualização e exclusão de alunos.
`ServicoEmpresa`	Cadastro, atualização e exclusão de empresas parceiras.
`ServicoVantagem`	Cadastro e listagem de vantagens.
`ServicoMoedas`	Envio de moedas, resgate de vantagem e crédito semestral; usa `NotificadorEmail`.
`NotificadorEmail` (interface)	Contrato de notificação: moedas recebidas e resgate de vantagem.
`ServicoEmailExterno`	Realiza `NotificadorEmail` usando o serviço de e-mail externo.

4.2 Regras de negócio no diagrama

Regra do enunciado	Representação no diagrama
Cada professor recebe 1.000 moedas por semestre, acumuláveis	Constante sublinhada `Professor.CREDITO_SEMESTRAL = 1000`; `receberCreditoSemestral()` soma ao saldo por `Conta.creditar`; `ServicoMoedas.creditarSemestre` aplica a todos os professores.

Regra do enunciado	Representação no diagrama
Para enviar, o professor precisa de saldo suficiente	`Conta.temSaldo` e `Conta.debitar` (que devolve `false` quando o saldo não cobre o valor).
A mensagem do envio é aberta e obrigatória	Atributo `EnvioMoedas.mensagem`, exigido por `ServicoMoedas.enviarMoedas`.
O aluno é notificado por e-mail ao receber moedas	`NotificadorEmail.notificarRecebimento`, chamado por `enviarMoedas`.
Professor e aluno consultam o extrato e o saldo	`extrato()` em `MembroInstituicao` (implementado por `Aluno` e `Professor`) e `Conta.consultarSaldo`.
Ao resgatar, o valor é descontado, e aluno e parceiro recebem e-mail com o mesmo código	`ServicoMoedas.resgatarVantagem`, `Conta.debitar`, `Cupom.gerar` e `NotificadorEmail.notificarResgate`.
Professor e aluno pertencem a uma instituição, que fica explícita	Associação `InstituicaoEnsino` 1 — 0..* `MembroInstituicao`.
Alunos, professores e empresas têm login e senha	`Usuario` é a superclasse dos três; `ServicoAutenticacao` valida o acesso.

4.3 Cardinalidades

Associação	Cardinalidade	Justificativa
`InstituicaoEnsino` — `MembroInstituicao`	1 — 0..*	Aluno e professor pertencem a exatamente uma instituição, e a instituição pode ter vários membros.
`MembroInstituicao` ◆ `Conta` (composição)	1 — 1	Todo aluno e todo professor tem uma única conta, que não existe sem ele.
`Professor` — `EnvioMoedas`	1 — 0..*	Cada envio tem um professor; o professor pode fazer vários envios.
`Aluno` — `EnvioMoedas`	1 — 0..*	Cada envio tem um aluno de destino; o aluno pode receber vários envios.
`Aluno` — `ResgateVantagem`	1 — 0..*	Cada resgate é de um aluno; o aluno pode resgatar várias vezes.
`Vantagem` — `ResgateVantagem`	1 — 0..*	Cada resgate é de uma vantagem; uma vantagem pode ser resgatada várias vezes.
`ResgateVantagem` ◆ `Cupom` (composição)	1 — 1	Cada resgate gera exatamente um cupom, com o código enviado nos dois e-mails.
`EmpresaParceira` ◆ `Vantagem` (composição)	1 — 0..*	A vantagem pertence a uma empresa. A empresa pode ter nenhuma vantagem ao se cadastrar, pois o cadastro da vantagem é um caso de uso à parte.
`ServicoMoedas` → `NotificadorEmail`	1 — 1	O serviço usa uma única implementação do notificador.

> O enunciado não impõe limites numéricos de quantidade (como um máximo de alunos). O único valor numérico é o crédito de 1.000 moedas por semestre, representado como constante. Há duas relações muitos-para-muitos, entre Professor e Aluno (via `EnvioMoedas`) e entre Aluno e Vantagem (via `ResgateVantagem`). As duas foram resolvidas com uma classe intermediária, pois cada ocorrência guarda data, valor e dados próprios.

4.4 Princípios de projeto aplicados

Princípio	Aplicação no modelo
Responsabilidade única (SRP)	Cada serviço trata de um assunto (autenticação, aluno, empresa, vantagem, moedas). `Conta` concentra as regras de saldo; `Cupom` concentra o código.
Aberto/fechado (OCP)	Novos tipos de transação entram como subclasses de `Transacao`; novos canais de aviso entram como novas realizações de `NotificadorEmail`.
Substituição de Liskov (LSP)	`Aluno` e `Professor` substituem `MembroInstituicao`, e o `extrato()` devolve `List<Transacao>` nos dois casos.
Segregação de interfaces (ISP)	`NotificadorEmail` tem apenas as duas operações que `ServicoMoedas` usa.
Inversão de dependência (DIP)	`ServicoMoedas` depende da interface `NotificadorEmail`, e não do envio concreto de e-mail.
Alta coesão e baixo acoplamento	Dados e regras comuns ficam em `MembroInstituicao` e `Conta`, sem repetição; as associações já expressam os vínculos, sem atributos duplicados.

5. Diagrama de componentes (arquitetura MVC)

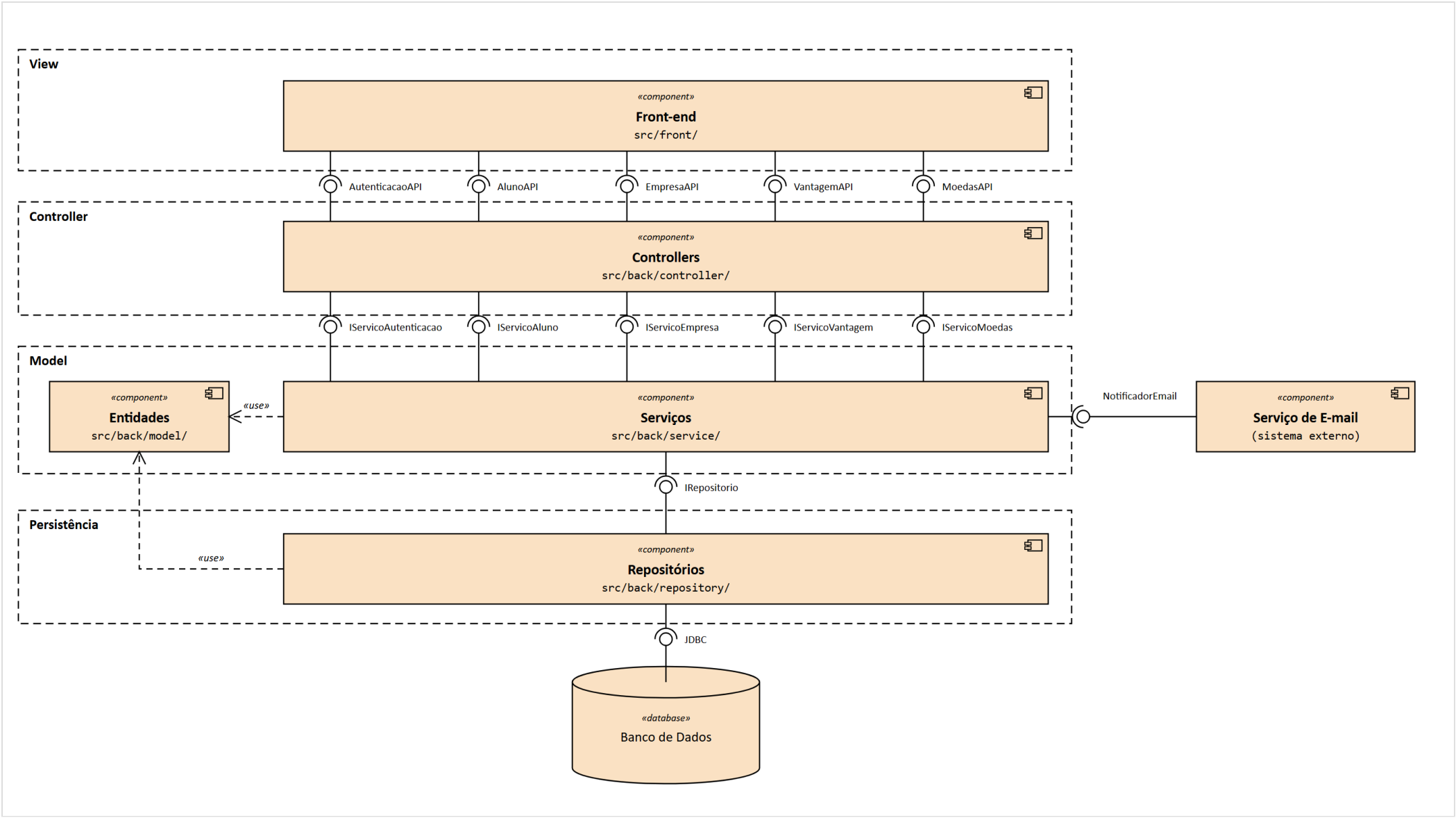


Figura 3 – Diagrama de componentes na arquitetura MVC, com interfaces fornecidas (bola) e requeridas (soquete)

O diagrama de componentes segue a arquitetura MVC exigida no enunciado, com a persistência em uma camada própria. Cada componente corresponde a uma pasta real do projeto, para que o modelo e o código fiquem alinhados. Os componentes representam responsabilidades arquiteturais e não classes individuais: por exemplo, o componente *Serviços* reúne as cinco classes de serviço do diagrama de classes.

5.1 Componentes

Componente	Camada MVC	Pasta	Responsabilidade
Front-end	View	`src/front/`	Telas e interação com o usuário; consome as APIs dos controllers.
Controllers	Controller	`src/back/controller/`	Recebem as requisições, chamam os serviços e devolvem a resposta.
Serviços	Model	`src/back/service/`	Regras de negócio: autenticação, aluno, empresa, vantagem e moedas.
Entidades	Model	`src/back/model/`	Classes do diagrama de classes, usadas pelos serviços e pelos repositórios.
Repositórios	Persistência	`src/back/repository/`	Acesso aos dados; a estratégia (ORM, DAO) será detalhada na Sprint 2.
Banco de Dados	Persistência	—	Armazenamento dos dados do sistema.
Serviço de E-mail	Externo	—	Sistema externo que envia os e-mails; `ServicoEmailExterno` é a classe que o usa.

5.2 Interfaces

Cada interface fornecida (bola) é ligada à interface requerida (soquete) do componente que a consome.

Interface	Fornecida por	Requerida por	Operações típicas
`AutenticacaoAPI`	Controllers	Front-end	`autenticar(login, senha)`
`AlunoAPI`	Controllers	Front-end	cadastrar, atualizar e excluir aluno
`EmpresaAPI`	Controllers	Front-end	cadastrar, atualizar e excluir empresa
`VantagemAPI`	Controllers	Front-end	cadastrar e listar vantagens
`MoedasAPI`	Controllers	Front-end	enviar moedas, resgatar vantagem e consultar extrato
`IServicoAutenticacao`	Serviços	Controllers	`autenticar(login, senha)`
`IServicoAluno`	Serviços	Controllers	`cadastrarAluno`, `atualizarAluno`, `excluirAluno`
`IServicoEmpresa`	Serviços	Controllers	`cadastrarEmpresa`, `atualizarEmpresa`, `excluirEmpresa`
`IServicoVantagem`	Serviços	Controllers	`cadastrarVantagem`, `listarVantagens`

Interface	Fornecida por	Requerida por	Operações típicas
`IServicoMoedas`	Serviços	Controllers	`enviarMoedas`, `resgatarVantagem`, `creditarSemestre`
`IRepositorio`	Repositórios	Serviços	salvar, buscar, listar e excluir (a detalhar na Sprint 2)
`JDBC`	Banco de Dados	Repositórios	acesso ao banco (a definir na Sprint 2: JDBC, ORM ou DAO)
`NotificadorEmail`	Serviço de E-mail	Serviços	`notificarRecebimento`, `notificarResgate`

5.3 Dependências

- As dependências seguem de cima para baixo: View → Controller → Model (serviços) → Persistência → Banco de Dados. O front-end nunca acessa o banco diretamente.
- *Serviços* e *Repositórios* usam as *Entidades* (setas tracejadas «use»).
- O *Serviço de E-mail* aparece como componente externo, pois está fora do controle direto da equipe. *Serviços* o requer por meio de `NotificadorEmail`.
- A estratégia de acesso a dados (ORM ou DAO) será definida na Sprint 2. A camada de persistência já aparece aqui, para a próxima sprint apenas detalhá-la.

6. Rastreabilidade entre os modelos

A tabela liga cada caso de uso (e a história correspondente) às classes e operações do diagrama de classes e aos componentes que o realizam, para mostrar que os quatro modelos são consistentes entre si.

Caso de uso e história	Classes e operações	Componentes
Autenticar-se	`ServicoAutenticacao.autenticar`, `Usuario.autenticar`	Controllers, Serviços
Cadastrar / atualizar / excluir aluno	`ServicoAluno`, `Aluno`, `InstituicaoEnsino`, `Conta`	Controllers, Serviços, Entidades, Repositórios
Consultar extrato do aluno	`Aluno.extrato`, `Conta.consultarSaldo`, `EnvioMoedas`, `ResgateVantagem`	Controllers, Serviços, Entidades
Trocar moedas por vantagem	`ServicoMoedas.resgatarVantagem`, `Conta.temSaldo/debitar`, `Cupom.gerar`, `NotificadorEmail.notificarResgate`	Controllers, Serviços, Serviço de E-mail
Enviar moedas a um aluno	`ServicoMoedas.enviarMoedas`, `Conta.temSaldo/debitar/creditar`, `EnvioMoedas`, `NotificadorEmail.notificarRecebimento`	Controllers, Serviços, Serviço de E-mail
Consultar extrato do professor	`Professor.extrato`, `Conta.consultarSaldo`	Controllers, Serviços, Entidades
Receber moedas do semestre	`ServicoMoedas.creditarSemestre`, `Professor.receberCreditoSemestral`, `CREDITO_SEMESTRAL`	Serviços, Entidades
Cadastrar / atualizar / excluir empresa	`ServicoEmpresa`, `EmpresaParceira`	Controllers, Serviços, Entidades, Repositórios
Cadastrar vantagem	`ServicoVantagem.cadastrarVantagem`, `Vantagem`	Controllers, Serviços, Entidades, Repositórios
Notificar aluno sobre moedas recebidas	`NotificadorEmail.notificarRecebimento`, `ServicoEmailExterno`	Serviço de E-mail
Enviar cupom de resgate	`NotificadorEmail.notificarResgate`, `ServicoEmailExterno`	Serviço de E-mail

7. Próximas etapas e referências

7.1 Próximas sprints

- **Sprint 2:** modelo entidade-relacionamento, definição e implementação da estratégia de acesso ao banco de dados (ORM, padrão DAO etc.) e versão inicial dos CRUDs de aluno e empresa parceira (front-end e comunicação com o back-end).
- **Sprint 3:** versão final dos CRUDs, apresentação da arquitetura e da camada de persistência e tutorial das tecnologias escolhidas.

Se o código exigir ajustes, os diagramas serão atualizados e gerados em novos arquivos (`sprint2-`, `sprint3-`), mantendo as versões anteriores.

7.2 Referências

- Enunciado do Laboratório 3 – Sistema de Moeda Estudantil. Profa. Milena Menezes Adão, PUC Minas, 2º semestre de 2026.
- Material da disciplina: A.04.1 – Diagrama de Componentes.
- Repositório do projeto: github.com/lucaslmchaves/sistema-moeda-estudantil (diagramas em `docs/diagramas/`).